Graph Preconditioning for High-Dimensional Fixed Effects Regression

Alexander Fischer[1] and Kristof Schröder[2]

Draft: June 2026

Abstract. The Method of Alternating Projections (MAP) is the canonical algorithm for estimating high-dimensional fixed-effect regressions. It is fast when absorbed factors are well connected, but converges slowly on sparse or nearly nested fixed-effect graphs, such as matched employer-employee panels where worker-firm mobility links separate worker and firm effects. The convergence behavior of MAP depends on the mobility pattern linking workers to firms, yet MAP only indirectly makes use of this information by iterating over one fixed effect at a time. That mobility pattern is, however, directly encoded in the matrix of worker-firm match counts, which together with the worker and firm count diagonals forms a graph Laplacian after a sign flip and admits sparse approximate Cholesky factorization. We propose a graph-preconditioned Krylov solver whose reusable preconditioner is built from factor-pair subproblems, such as worker-firm and worker-year pairs, that use the graph directly. In our benchmarks, building the factor-pair preconditioner costs more than it saves on well-connected designs. Graph preconditioning reduces total runtime on poorly connected worker-firm graphs, including those produced by strong sorting.

Keywords: high-dimensional fixed effects; alternating projections; preconditioning; matched employer-employee data; computational econometrics

JEL codes: C55; C63; C81; C87; J31

1. Introduction

Fixed-effect regressions are ubiquitous in applied econometrics: according to Goldsmith-Pinkham (2026), roughly half of published research in top economics and finance journals mentions "fixed effects". They appear throughout all fields of applied economics: labor economists use worker and firm fixed effects to separate worker heterogeneity from firm wage premia; health economists study physician practice styles with individual-physician and region fixed effects in mover designs; and education researchers study models with school, student, teacher, or student-teacher fixed effects.

The standard computational starting point for estimating these regressions efficiently is the Frisch-Waugh-Lovell (FWL) theorem (Frisch and Waugh 1933; Lovell 1963). FWL reduces the fixed-effect estimation problem to "residualizing" the outcome and every regressor of interest against the fixed effects, and then to running a low-dimensional regression on the residualized variables.

The workhorse method for these fixed-effect residualizations is the Method of Alternating Projections (MAP), also known as iterative demeaning or the "Zig-Zag" algorithm (Guimaraes and Portugal 2010;

[1]trivago
[2]appliedAI Institute for Europe gGmbH

Gaure 2013). Most leading software implementations of fixed-effect regression, such as `reghdfe` in Stata (Correia 2014; 2017), `fixest` (Bergé et al. 2026) in R, or PyFixest in Python (PyFixest Developers 2026), use MAP or MAP-based variants with acceleration.

Because the AKM worker-firm model is among the most prominent applications of high-dimensional fixed effects, we use a worker-firm-year panel based on Abowd et al. (1999) as the running example in this paper. The method of alternating projections cycles through the fixed-effect dimensions one at a time: it first subtracts worker means, then firm means, then year means, and repeats until convergence. The coupling between fixed effects is never used directly; the cross-fixed effects information is transmitted only indirectly through the residual that each step hands to the next. How fast MAP converges therefore depends on how quickly information about this coupling can propagate from one update to the next.

Fixed effects and their coupling form a graph structure. In a worker-firm panel, movers create paths between firms, while stayers add observations without connecting firms to one another. When this graph is sparse or poorly connected, some fixed-effect directions are nearly collinear, and MAP needs many iterations to propagate information across it before converging. The same graph features that determine whether worker and firm effects are identified also govern MAP convergence. These features include worker mobility, sorting, and segmentation into disconnected labor markets with thin bridges, such as public- and private-sector employment when workers only rarely move between the two sectors.

The graph structure of the fixed effects can be algebraically encoded in the off-diagonal blocks of the fixed-effect Gramian (Correia 2017). These off-diagonal blocks record co-occurrences among the fixed effects: which workers work at which firms, which physicians practice in which regions, or which families move across counties.

In this paper, we propose a new preconditioner that directly encodes the co-occurrence graph. A preconditioner is a cheap approximation to the system being solved; supplied to an iterative solver, it reduces the number of iterations without changing the solution. We build ours from local factor-pair subproblems that use the graph structure of the Gramian: for example, a worker-firm subproblem incorporates the observed links between workers and firms. A preconditioner is only useful if it approximates the inverse of the co-occurrence Gramian at low cost. We obtain such an approximation by exploiting the fact that, after a sign change, each factor-pair block is a graph Laplacian, which admits sparse approximate inverses following ideas from the Laplacian-solver literature (Spielman and Teng 2014; Gao et al. 2025). The preconditioned system is then solved with a Krylov solver, an iterative method for large linear systems.[3] Poorly connected graphs require more MAP iterations, and graph preconditioning is faster in those cases. On well-connected graphs, its setup cost outweighs the iteration savings; MAP and diagonally preconditioned Krylov methods run faster.

The rest of the paper is organized as follows. Section 2 sets up the fixed-effect absorption problem, and Section 3 introduces the AKM model as our running example. Section 4 develops the graph structure of the fixed-effect Gramian, and Section 5 connects this structure to the convergence behavior of MAP. Section

[3]The Julia implementation of fixed-effect regression, `FixedEffectModels.jl` (FixedEffectModels.jl Developers 2026), uses the same Krylov solver as we do - LSMR (Fong and Saunders 2011) - but only with diagonal preconditioning, which ignores the off-diagonal co-occurrence structure entirely; our contribution is the preconditioner, not the use of LSMR for the outer iteration.

6 builds up the factor-pair Schwarz preconditioner, starting from a general discussion of preconditioning and culminating in the construction of the graph-based preconditioner. Section 7 reports benchmarks on runtime, memory, and numerical equivalence; Section 8 describes the software through which the new algorithm is available; and Section 9 concludes.

2. Absorbing Fixed Effects[4]

We focus on the linear model

$$y = X\beta + D\alpha + \varepsilon, \tag{1}$$

where X contains the regressors of interest, D is the fixed-effect design matrix, and α collects the fixed-effect coefficients. In high-dimensional applications D may have hundreds of thousands or millions of columns, and forming or inverting the full system in $[X \quad D]$ might prove computationally infeasible. Fortunately, via the Frisch-Waugh-Lovell (FWL) theorem, we can compute $\hat{\beta}$ without ever forming $[X \quad D]$ or inverting its cross product.

FWL reduces the computation of $\hat{\beta}$ to two steps. In step one, we residualize the outcome and each covariate against the fixed effects: we regress y on D and keep the residual $\tilde{y}$, and we do the same for every column of X. Denoting M_D as the linear operator that sends a variable to this residual, so that $\tilde{y} = M_D y$ and $\tilde{X} = M_D X$, the coefficient of interest is recovered by regressing the residualized outcome on the residualized covariates,

$$\tilde{y} = M_D y, \quad \tilde{X} = M_D X, \quad \hat{\beta} = \left(\tilde{X}' W \tilde{X}\right)^{-1} \tilde{X}' W \tilde{y}, \tag{2}$$

where W is a diagonal matrix of weights. With one covariate, the procedure consists of three regressions: y on D, x on D, and $\tilde{y}$ on $\tilde{x}$. FWL implies that the slope in the last regression equals the coefficient on x in the full regression of y on x and D.

The residualization step is the weighted least-squares projection of the outcome and each covariate in X onto the column space of the fixed-effect dummy matrix D. For a right-hand side μ, either y or one column of X, we solve

$$\hat{\alpha}_\mu \in \arg\min_\alpha \| \, D\alpha - \mu \, \|_W^2, \tag{3}$$

and the residual is $\tilde{\mu} = \mu - D\hat{\alpha}_\mu$. The first-order condition for Equation 3,

$$D'W\left(D\hat{\alpha}_\mu - \mu\right) = 0, \quad \text{equivalently} \quad G\hat{\alpha}_\mu = D'W\mu, \quad G = D'WD, \tag{4}$$

determines the fitted value $D\hat{\alpha}_\mu$ and the residual $\tilde{\mu}$, even though $\hat{\alpha}_\mu$ itself is not unique. With a full set of fixed-effect dummies, the columns of D are linearly dependent. To report individual fixed effects, one must choose a normalization, usually by dropping reference categories. The normalization does not change the fitted value, the residual, or the FWL slope.[5]

Each FWL residualization uses the same coefficient matrix G; only the right-hand side changes as we move from the outcome to the covariates. The cost of residualization therefore depends on the structure of the Gramian G. In the next section, we illustrate this structure with the AKM worker-firm model and explain

[4] Researchers employ several names for this operation: "absorbing fixed effects", "demeaning", "residualizing", or applying the "within transformation". We use these terms interchangeably throughout.

[5] Throughout, inverses are taken after normalizing each connected component.

why fixed-effect designs have a direct graph interpretation. Sections 5–6 then return to the algorithms and show how the structure of the Gramian and its associated graph governs MAP convergence, and explain how we use it to construct the factor-pair Schwarz preconditioner.

3. A Running Example: The AKM Model

The AKM model of Abowd et al. (1999) introduces the worker-firm setting used throughout the paper. It separates persistent worker heterogeneity from firm wage premia using workers who move across firms. We write the AKM regression equation as

$$y_{it} = \alpha_i + \psi_{J(i,t)} + \varphi_t + x'_{it}\beta + \varepsilon_{it}, \tag{5}$$

where α_i is a worker fixed effect, $\psi_{J(i,t)}$ is the fixed effect for the firm employing worker i at time t, and φ_t is a time fixed effect.

The AKM specification has a natural graph representation. Workers and firms are nodes in a bipartite graph. Each row identifies a worker and the firm that employs them; repeated observations of the same match add to its count. Worker moves induce a firm-to-firm graph, in which two firms are connected when at least one worker is observed at both firms. Although stayers - workers who never change their employer - add observations to existing worker-firm links, they do not create bridges between firms. Year effects enter as a third, low-dimensional factor observed on the same worker-firm records. Figure 1 contrasts a well-connected mobility graph with one that fragments under strong sorting.

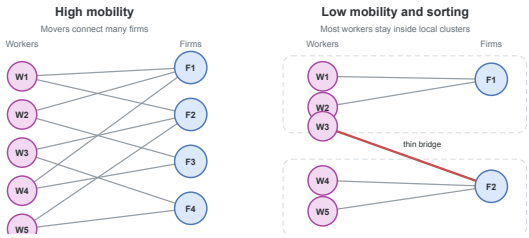

Figure 1: Worker-firm graph connectivity. When mobility is high, many paths connect firms. With low mobility and strong sorting, the graph breaks into nearly separate clusters joined only by narrow bridges.

Mobility links determine both identification and computational difficulty. Within a connected set, movers provide the comparisons that separately identify worker and firm fixed effects. A worker observed at only one firm provides no such comparison: a high wage could reflect an unusually productive worker, a high-wage firm, or both. Without worker moves, these components are not separately identified. Movers observed across firms with different wage profiles help attribute wage variation to worker effects or firm premia. If a worker earns high wages across several firms, the comparison points toward a worker effect; if many different workers earn higher wages at the same firm, it points toward a firm premium. The more such cross-firm comparisons the data contain, especially across otherwise different firms, the easier it is to identify worker and firm premia. In the graph, these moves are exactly the edges that connect firms; additional moves of workers across firms add worker-firm links to the graph.

4. The Graph Structure of the Gramian

The bipartite graph of worker and firm connections introduced in Section 3 has an algebraic representation in the block structure of the Gramian $G = D'WD$ (Correia 2017). Suppose that the columns of D are ordered as worker levels, firm levels, and year levels. Then

$$G = \begin{pmatrix} G_{WW} & C_{WF} & C_{WY} \\ C_{(WF)'} & G_{FF} & C_{FY} \\ C_{(WY)'} & C_{(FY)'} & G_{YY} \end{pmatrix}. \tag{6}$$

The diagonal blocks G_{WW}, G_{FF}, and G_{YY} contain weighted counts for workers, firms, and years. An observation belongs to one level of each factor, so these blocks are diagonal; solving them requires only division by group counts.

The off-diagonal blocks are cross-tabulations: the worker-firm block C_{WF} records how often worker i is observed at firm j, and the worker-year and firm-year blocks have analogous interpretations.

As a small example, we construct a worker-firm panel and populate its Gramian. For simplicity, we ignore any regression weights and set $W = I$.

Obs.	Worker	Firm	Year	y
1	W_1	F_1	Y_1	3.2
2	W_1	F_2	Y_2	4.1
3	W_2	F_1	Y_1	2.8
4	W_2	F_1	Y_2	3.9
5	W_3	F_2	Y_1	5.0
6	W_3	F_2	Y_2	4.5

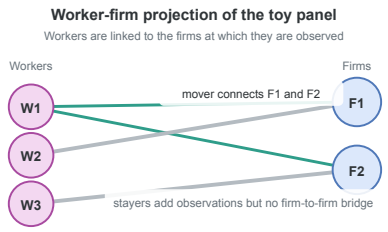

Figure 2: Worker-firm projection of the example panel. Worker W_1 is a mover; workers W_2 and W_3 are stayers.

Figure 2 plots the worker-firm projection of this panel. Worker W_1 is observed at both F_1 and F_2 and creates a link between the two firms; in AKM terms, W_1 is a mover. Worker W_2 stays at F_1 for two periods, and W_3 stays at F_2 for two periods. Both are stayers.

The diagonal blocks are count matrices. In this example, each worker is observed twice, each firm three times, and each year three times, so

$$G_{WW} = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{pmatrix}, \quad G_{FF} = \begin{pmatrix} 3 & 0 \\ 0 & 3 \end{pmatrix}, \quad G_{YY} = \begin{pmatrix} 3 & 0 \\ 0 & 3 \end{pmatrix}. \tag{7}$$

The off-diagonal blocks are cross-tabulations between factors. The worker-firm block is

$$C_{WF} = \begin{pmatrix} 1 & 1 \\ 2 & 0 \\ 0 & 2 \end{pmatrix}. \tag{8}$$

The first row indicates that worker W_1 appears once at firm F_1 and once at firm F_2. Workers W_2 and W_3 are stayers, appearing twice at F_1 and F_2, respectively. The worker-year block is

$$C_{WY} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{pmatrix}. \tag{9}$$

Each worker is observed once in each year. The firm-year block is

$$C_{FY} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}. \tag{10}$$

Firm F_1 appears twice in year Y_1 and once in year Y_2; firm F_2 has the opposite pattern. Combining the diagonal count blocks and the off-diagonal cross-tabulation blocks yields the full Gramian.

With column order $(W_1, W_2, W_3, F_1, F_2, Y_1, Y_2)$, the full Gramian is

$$G = \left(\begin{array}{ccc|cc|cc} 2 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 2 & 0 & 2 & 0 & 1 & 1 \\ 0 & 0 & 2 & 0 & 2 & 1 & 1 \\ \hline 1 & 2 & 0 & 3 & 0 & 2 & 1 \\ 1 & 0 & 2 & 0 & 3 & 1 & 2 \\ \hline 1 & 1 & 1 & 2 & 1 & 3 & 0 \\ 1 & 1 & 1 & 1 & 2 & 0 & 3 \end{array}\right). \tag{11}$$

The worker-firm submatrix stores the bipartite graph algebraically. The diagonal entries are worker and firm counts, and the entries of C_{WF} are edge multiplicities between workers and firms. After flipping the sign of C_{WF}, this submatrix is a graph Laplacian: its off-diagonal entries are non-positive, and every row sums to zero because each diagonal count cancels the off-diagonal observation counts in the same row. For a worker, the diagonal entry is the number of observations for that worker, while the off-diagonal entries show how those observations are distributed across firms; firm rows have the analogous interpretation with observations summed over workers.

$$L_{WF} = \left(\begin{array}{ccc|cc} 2 & 0 & 0 & -1 & -1 \\ 0 & 2 & 0 & -2 & 0 \\ 0 & 0 & 2 & 0 & -2 \\ \hline -1 & -2 & 0 & 3 & 0 \\ -1 & 0 & -2 & 0 & 3 \end{array}\right). \tag{12}$$

The same Laplacian construction applies to any pair of fixed effects, and the preconditioner of Section 6 builds on these pairwise Laplacians. Before turning to it, however, we introduce the method of alternating projections, which avoids forming the full Gramian G by working only on the diagonal worker, firm, and year blocks.

5. Alternating Projections and Graph Connectivity

The workhorse algorithm for multi-way fixed effects is the Method of Alternating Projections (MAP), also referred to as iterative demeaning or the "zig-zag" algorithm (Guimaraes and Portugal 2010; Gaure 2013). Many packages employ MAP or its variants, frequently combined with accelerations (Berge 2018; Correia 2017), such as the Irons-Tuck extrapolation used by `fixest` (Irons and Tuck 1969; Bergé et al.

2026). Sections 3 and 4 introduced the fixed-effect graph and its algebra; this section turns to MAP itself and shows how that graph geometry governs its convergence rate.

MAP solves the FWL residualization problem by iterating over one fixed effect at a time. In the worker-firm-year model, MAP first subtracts worker means from the current residual, then firm means from the updated residual, then year means, and so on until convergence.

We write the FWL normal equations (see Equation 4) in block form with $D = [D_W \quad D_F \quad D_Y]$ as

$$\begin{pmatrix} G_{WW} & C_{WF} & C_{WY} \\ C_{(WF)'} & G_{FF} & C_{FY} \\ C_{(WY)'} & C_{(FY)'} & G_{YY} \end{pmatrix} \begin{pmatrix} \alpha_W \\ \alpha_F \\ \alpha_Y \end{pmatrix} = \begin{pmatrix} D_{W'}W\mu \\ D_{F'}W\mu \\ D_{Y'}W\mu \end{pmatrix}. \tag{13}$$

Equivalently,

$$G_{WW}\alpha_W + C_{WF}\alpha_F + C_{WY}\alpha_Y = D_{W'}W\mu, \tag{14}$$

$$C_{(WF)'}\alpha_W + G_{FF}\alpha_F + C_{FY}\alpha_Y = D_{F'}W\mu, \tag{15}$$

$$C_{(WY)'}\alpha_W + C_{(FY)'}\alpha_F + G_{YY}\alpha_Y = D_{Y'}W\mu. \tag{16}$$

Each equation can be rearranged to express one block of effects conditional on the others. Using $C_{WF} = D_{W'}WD_F$ and $C_{WY} = D_{W'}WD_Y$ to factor $D_{W'}W$ out of the right-hand side, the first equation becomes

$$G_{WW}\alpha_W = D_{W'}W(\mu - D_F\alpha_F - D_Y\alpha_Y). \tag{17}$$

Because G_{WW} is a diagonal matrix whose entries are workers' total observation weights, solving for α_W divides each worker's weighted partial residual by its total observation weight. We apply the same rearrangement to the second equation to obtain the firm equation

$$G_{FF}\alpha_F = D_{F'}W(\mu - D_W\alpha_W - D_Y\alpha_Y), \tag{18}$$

and the year equation is analogous. Because G_{WW}, G_{FF}, and G_{YY} are all diagonal, each of the three equations is solved by computing a weighted group mean in a single pass over observations.

MAP uses this diagonal structure iteratively. Holding the other effects fixed, it updates the worker effects from the current partial residual, then repeats the same step for firms and years. Each sweep therefore cycles through the fixed-effect dimensions, subtracting the weighted group mean of the current partial residual for the factor being updated.

The cross-tabulation blocks C_{WF}, C_{WY}, and C_{FY} enter the algorithm only indirectly. For instance, the worker update is computed from the partial residual $\mu - D_F\alpha_F - D_Y\alpha_Y$, while the firm update is computed from $\mu - D_W\alpha_W - D_Y\alpha_Y$. Worker-firm, worker-year, and firm-year links are thus not solved as coupled subproblems; their effect propagates through the residual that one block update transmits to the next.

This strategy is effective when the graph is well connected. In a high-mobility worker-firm panel, many workers move across firms, so that worker and firm effects can be compared through many overlapping employment histories. A high wage at one firm can then be related to wages earned by the same workers at other firms, and a worker update rapidly alters the information available to the next firm update, and vice versa.

When mobility is sparse, sorting is strong, or one factor is nearly nested in another, MAP may require many sweeps because mover comparisons enter only through repeated residual updates. Each sweep is cheap: the diagonal block solves are one-pass group means.

The same graph perspective gives a simple diagnostic for MAP difficulty in a fixed-effect structure. For any pair of fixed effects (q, r), we form the normalized cross-tabulation $H_{qr} = G_{qq}^{-\frac{1}{2}} C_{qr} G_{rr}^{-\frac{1}{2}}$ and let $\rho_{qr} = \sigma_2\big(H_{qr}\big)^2$ be the square of its largest nontrivial singular value, equivalently the largest nontrivial eigenvalue of $H_{(qr)'} H_{qr}$. Within a connected component, the largest singular value is always one, regardless of how well connected the component is. We use the next-largest singular value and report the spectral gap $1 - \rho_{qr}$ in the benchmarks below. This gap measures how well connected the factor-pair graph is.[6] Gaps near zero signal sparse mobility or near-nesting, the settings in which MAP converges slowly; larger gaps indicate better connected factor-pair graphs. For the worker-firm example of Section 4, the gap is $\frac{1}{3}$.[7]

6. The Factor-Pair Schwarz Preconditioner

6.1. Preconditioners

The previous section attributed MAP's slow convergence to thin connections in the fixed-effect graph. A solver that receives this connectivity information directly should converge faster. MAP has no natural way to accept it: its update rule is fully determined by the list of absorbed factors, and the cross-tabulations enter only through the residual that one update passes to the next. Krylov solvers, by contrast, take an additional input, the preconditioner, and this input is where we place the graph structure. We therefore replace factor-by-factor demeaning via MAP with LSMR (Fong and Saunders 2011), an iterative least-squares algorithm that improves an initial guess through repeated residual corrections. The change of solver gains little by itself: a poorly conditioned Gramian G slows LSMR just as a poorly connected graph slows MAP. The core acceleration stems from building a good preconditioner, which we focus on in the remainder of this section.

How fast LSMR converges depends on the conditioning of G. When G is well conditioned, LSMR shrinks every component of the residual at a comparable rate. When G is poorly conditioned, LSMR removes some components after a few iterations, whereas components that correspond to weak links, sparse mobility, or near nesting in the fixed-effect graph shrink much more slowly; the iteration then spends most of its steps on these slow components. A preconditioner counteracts this imbalance: it changes the coordinates of the linear system so that slow and fast components decay at more similar rates, without changing the least-squares solution.

The ideal preconditioner is G^{-1}. If $M^{-1} = G^{-1}$, then the preconditioned operator is

$$M^{-1}G = G^{-1}G = I. \tag{19}$$

[6] When the graph has more than one connected component, we compute the gap on each component and report the smallest, together with its observation share.

[7] In that example, $G_{WW} = \mathrm{diag}(2,2,2)$, $G_{FF} = \mathrm{diag}(3,3)$, and $C_{WF} = \begin{pmatrix} 1 & 1 \\ 2 & 0 \\ 0 & 2 \end{pmatrix}$, so $H_{WF} = G_{WW}^{-\frac{1}{2}} C_{WF} G_{FF}^{-\frac{1}{2}} = \frac{1}{\sqrt{6}} \begin{pmatrix} 1 & 1 \\ 2 & 0 \\ 0 & 2 \end{pmatrix}$. The graph is connected, and $H_{(WF)'} H_{WF} = \frac{1}{6} \begin{pmatrix} 5 & 1 \\ 1 & 5 \end{pmatrix}$ has eigenvalues 1 and $\frac{2}{3}$. After dropping the unit eigenvalue, $\rho_{WF} = \frac{2}{3}$ and the gap is $\frac{1}{3}$.

In exact arithmetic, the Krylov iteration would then recover the solution after one correction, because the system has no slow directions left to remove. To form G^{-1} we would have to solve the fixed-effect normal equations themselves, so the identity case serves as a benchmark rather than an implementable preconditioner. A useful preconditioner must approximate enough of G^{-1} to remove the slow directions of the iteration, while its construction and repeated application must amortize over the iterations it saves.[8]

6.2. From Block Elimination to the Diagonal Preconditioner

The block structure of G^{-1} shows which parts of the ideal inverse a feasible preconditioner should retain. For the worker-firm-year AKM model, the Gramian has the block form

$$G = \begin{pmatrix} G_{WW} & C_{WF} & C_{WY} \\ C_{(WF)'} & G_{FF} & C_{FY} \\ C_{(WY)'} & C_{(FY)'} & G_{YY} \end{pmatrix}, \tag{20}$$

with diagonal weighted-count blocks G_{WW}, G_{FF}, G_{YY} and off-diagonal cross-tabulations C_{WF}, C_{WY}, C_{FY}. Block inversion via the Schur complement shows how each off-diagonal block enters G^{-1}. For the two-factor block

$$G_2 = \begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix}, \tag{21}$$

eliminating the worker effects gives the firm-side Schur complement

$$S = G_{FF} - C_{(WF)'} G_{WW}^{-1} C_{WF}. \tag{22}$$

The expanded block inverse is given in Appendix A. Applying it requires solves with G_{WW} and S; C_{WF} appears only in matrix products. G_{WW} is diagonal, so G_{WW}^{-1} is a division by weighted worker counts. The Schur complement S, by contrast, is the firm-side mobility system that remains after eliminating workers. At the scale of modern worker-firm register data, solving this system is expensive: exact factorization creates many additional nonzero entries, separate connected components require separate normalizations, and weak mobility makes the remaining directions slow to resolve. S^{-1} therefore carries almost all the cost of the solve. Block inversion via Schur complements generalizes to three factors: the closed form has more terms, but every block of G^{-1} still depends jointly on the cross-tabulations C_{WF}, C_{WY}, C_{FY}.

The coarsest approximation to G^{-1} keeps only the diagonal count inverses and drops the Schur-complement corrections,

$$M_{\text{diag}}^{-1} = \operatorname{diag}(G_{WW}^{-1}, G_{FF}^{-1}, G_{YY}^{-1}). \tag{23}$$

This preconditioner is a single division by weighted level counts. It is the diagonal preconditioner used with LSMR in `FixedEffectModels.jl` (Fong and Saunders 2011; FixedEffectModels.jl Developers 2026). Diagonal scaling encodes how many observations a level carries, but not how that level connects to the rest of the labor market. Those counts can already be useful when employment is concentrated in a few large firms, such as Novo Nordisk in Denmark or Samsung in South Korea, because the size differences alone remove an important source of scale variation. What diagonal scaling does not record is whether

[8] LSMR never forms $M^{-1}G$ explicitly, nor G itself. The iteration requires only products with D and D' and applications of M^{-1}, supplied as linear operators.

workers at those firms link them broadly to other firms or remain concentrated in a narrow corner of the mobility graph.

6.3. The Factor-Pair Schwarz Approximation

Additive Schwarz preconditioning adds this missing pairwise connectivity without solving the full three-factor system (Xu 1992; Toselli and Widlund 2005). It splits the fixed-effect problem into smaller overlapping pair problems. In the AKM case, one problem contains workers and firms, another contains workers and years, and a third contains firms and years. The worker-firm problem moves residual information along observed employment links; the other two pair problems do the same for worker-year and firm-year links. We then combine the three pair corrections in the full coefficient space. The preconditioner therefore gives the Krylov iteration the main pairwise channels of the Gramian, while the outer iteration handles the remaining three-way coupling.

To make the local subproblems concrete, we first consider the worker-firm pair. Its local problem is exactly the two-factor block from the Schur calculation,

$$\begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix}. \tag{24}$$

Figure 3 places this worker-firm pair block beside the single diagonal block solved by a factor-level MAP update.

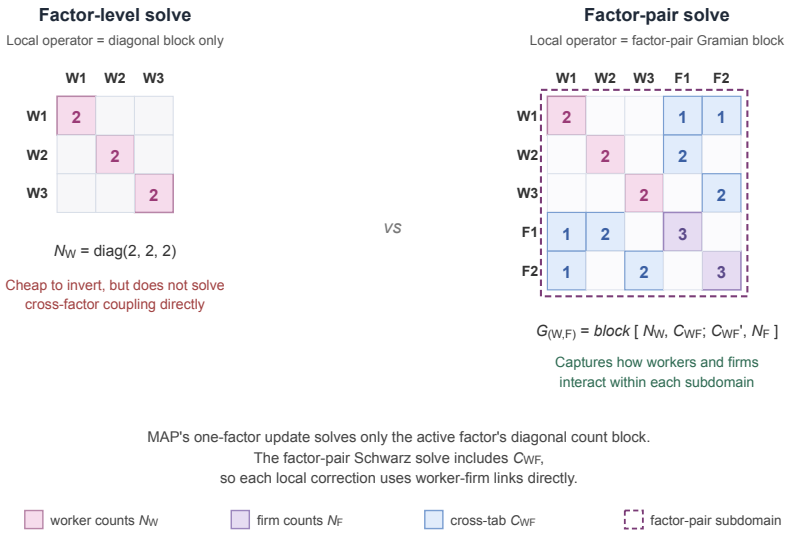

Figure 3: Local operator used by a factor-level MAP update (left) versus the factor-pair Schwarz solve (right), shown on the example worker-firm panel of Section 4. The factor-level block is the diagonal $G_{WW} = \mathrm{diag}(2, 2, 2)$. The factor-pair block adds the firm count block $G_{FF} = \mathrm{diag}(3, 3)$ and the cross-tabulation C_{WF} in its off-diagonal positions; the dashed outline marks the worker-firm subdomain on which the local Schwarz solve operates.

The Schur complement incorporates C_{WF}, so the local correction uses the worker-firm mobility pattern that diagonal scaling discards. To place this correction inside the three-factor problem, let R_{WF} select the worker and firm entries from the full coefficient vector $\alpha = [\alpha_W; \alpha_F; \alpha_Y]$; its transpose $R_{(WF)'}$ places the resulting correction back into the full vector. The diagonal matrix $\tilde{D}_{WF}$ contains the weights that

split levels across the pair problems in which they appear; we call these entries partition-of-unity weights. The exact worker-firm contribution is

$$P_{WF}^{-1} = R_{(WF)'}\tilde{D}_{WF}\begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix}^{-1}\tilde{D}_{WF}R_{WF}. \tag{25}$$

The worker-year and firm-year contributions P_{WY}^{-1} and P_{FY}^{-1} are built the same way from G_{WW}, C_{WY}, G_{YY} and G_{FF}, C_{FY}, G_{YY}. With three factors each level appears in exactly two pair problems. Because the weights act on both sides of each pair contribution, we set them so that the squared weights on a shared level sum to one; a level in two pairs then carries $\frac{1}{\sqrt{2}}$. The exact factor-pair Schwarz preconditioner is the sum of these three contributions,

$$P^{-1} = P_{WF}^{-1} + P_{WY}^{-1} + P_{FY}^{-1}. \tag{26}$$

The three terms include the worker-firm, worker-year, and firm-year cross-tabulations separately. They do not solve the simultaneous worker-firm-year problem; that remaining coupling, together with the error introduced by splitting shared levels across pairs, is left to the outer LSMR iteration.

6.4. Approximate Pair Solves via Graph Laplacians

For P^{-1} to serve as the operator M^{-1} inside LSMR, each pair contribution must be computed without solving a large dense system. For factor pairs with few levels, we invert the pair block directly. The example in Section 4 requires only a 4×4 local solve. In modern worker-firm register data, a pair may contain hundreds of thousands of levels on each side, making direct factorization impractical. We use the graph-Laplacian form of the pair block.

For a worker-firm pair, the local Schwarz step solves the pair-Gramian system

$$\begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix} x = u, \tag{27}$$

where u is the weighted worker-firm part of the current Krylov residual. This block is not a graph Laplacian, because its off-diagonal entries C_{WF} are non-negative. A sign flip removes the obstacle. Let $T_{WF} = \mathrm{diag}(I_W, -I_F)$ flip the sign of the firm entries, so that $T_{WF}^2 = I$. Conjugation by T_{WF} gives

$$L_{WF} = T_{WF}\begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix}T_{WF} = \begin{pmatrix} G_{WW} & -C_{WF} \\ -C_{(WF)'} & G_{FF} \end{pmatrix}, \tag{28}$$

a weighted bipartite graph Laplacian: symmetric, with non-positive off-diagonals and zero row sums. Because $T_{WF}^2 = I$, the pair-Gramian solve follows from the Laplacian solve by the same flip on each side,

$$\begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix}^{-1} = T_{WF}L_{WF}^{-1}T_{WF}. \tag{29}$$

We solve the worker-firm block by reversing the signs of the firm entries in the right-hand side, solving one Laplacian system, and reversing the firm signs in the solution. A Laplacian solve requires the right-hand side to sum to zero within each connected component. The implementation subtracts the mean within each component before applying the local solve (Appendix A).

For preconditioning, this local solve need not be exact. The outer LSMR iteration refines any error left by the preconditioner. We therefore approximate the Laplacian solve using sparse approximate Cholesky factorizations from the Laplacian-solver literature (Spielman and Teng 2014; Gao et al. 2025). An exact Cholesky factorization of the pair block creates fill-in: eliminating a worker inserts entries linking every pair of distinct firms that worker visited, and these entries accumulate as the elimination proceeds. In the worst case, the cost approaches the order k^3 operations and order k^2 memory of a dense factorization of a k-level system. For large components, the implementation samples the fill-in edges that would be created during elimination instead of storing all of them. It applies randomized approximate Cholesky to the resulting sparse Laplacian (Spielman and Teng 2014; Gao et al. 2025). The expected factorization cost is linear in the number of observed worker-firm links, apart from logarithmic factors. We denote the corresponding approximate solve in worker-firm coordinates by A_{WF}.

The same construction yields A_{WY} and A_{FY} for the other two pairs. We substitute these approximate inverses into the Schwarz sum to obtain the implemented preconditioner,

$$M^{-1} = \sum_{(q,r)} R_{(qr)'} \tilde{D}_{qr} A_{qr} \tilde{D}_{qr} R_{qr}. \tag{30}$$

In the worker-firm-year case each factor receives a diagonal contribution from its two pair subdomains and each off-diagonal correction from the corresponding pair.

There are two approximations: P^{-1} replaces the full three-factor inverse with pair solves, and M^{-1} replaces each exact pair solve with A_{qr}. They may change the number of LSMR iterations, but not the fitted residuals; LSMR still solves the original least-squares problem to the requested tolerance.

6.5. Implementation

Figure 4 shows the construction. For each pair of absorbed factors, a sign change converts the local Gramian to a graph Laplacian. The implementation handles large connected components with approximate Schur reduction and sparse approximate Cholesky, then maps the corrections back to the full coefficient vector and combines them using partition-of-unity weights.

The outer LSMR iteration uses this preconditioner to shape its search directions (Fong and Saunders 2011; Arridge et al. 2014; Yang et al. 2024). Once built, the same preconditioner can residualize the outcome and every covariate. Appendix A lists the setup and application steps.

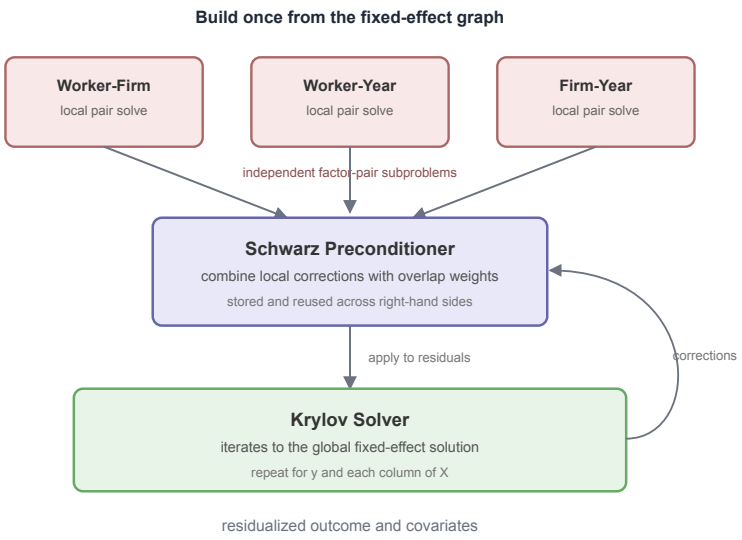

Figure 4: Construction of the factor-pair preconditioner. For large pair blocks, the algorithm approximates the Schur complement and factors the reduced Laplacian with approximate Cholesky. The weighted pair corrections form the preconditioner used by LSMR.

7. Benchmarks

Graph preconditioning should help most when weak connectivity slows MAP. We test this in controlled and public benchmark designs.

The main runtime tables use package-level regression APIs, matching the public PyFixest benchmark suite, rather than isolated demeaning kernels. Each timing covers the full regression: model setup, construction of the fixed-effect representation, residualization of the outcome and covariates, and coefficient estimation. Separate tables report memory use and coefficient agreement.

The runtime tables also report the spectral gap for the relevant factor pair and the observation share of the component attaining it. Smaller gaps are associated with slower MAP convergence. With three or more fixed effects, the gap remains a pairwise diagnostic, not a convergence bound for the full system.

We compare two MAP backends and two Krylov solvers:

Backend	Package	Algorithm
`rust-map`	PyFixest	Unaccelerated Rust MAP.
`fixest`	R `fixest`	Accelerated MAP with Irons-Tuck and other optimizations (Bergé et al. 2026).
`FEM.jl`	`FixedEffectModels.jl`	Diagonally preconditioned LSMR (Fong and Saunders 2011; FixedEffectModels.jl Developers 2026).
`within`	PyFixest	LSMR with factor-pair Schwarz preconditioning.

Each CPU result is based on three trials run on an Apple M4 Mac mini with 10 CPU cores and 16 GB of memory running macOS 15.3.1. We report the median among trials that converged. If only one or two trials converge, the table gives that count. `failed` means that none of the three trials converged before reaching the iteration cap.

Stopping rules differ across packages. PyFixest's MAP uses a nominal 10^{-6} tolerance and a 10,000-iteration cap, while `within` uses 10^{-8} and 1,000 iterations. These numbers are not directly comparable because the packages monitor different convergence quantities. We retain each package's default rule.

We omit `reghdfe` (Correia 2014; 2017) because Stata is not open source and we lack a license. Like `fixest`, it uses accelerated MAP.

7.1. Runtime Benchmarks

7.1.1. Controlled Synthetic Benchmarks: AKM Mobility and Sorting

These AKM panels hold the rest of the DGP fixed while mobility changes. Lower mobility leaves fewer workers connecting firms, which slows MAP.

Mobility benchmark ($n = 1$M).

Scenario	Gap (share)	rust-map	fixest	FEM.jl	within
akm_mobility_1	0.373 (1.00)	0.337s	0.472s	0.361s	0.690s
akm_mobility_2	0.0709 (1.00)	1.23s	0.912s	0.627s	0.479s
akm_mobility_3	5.02×10^{-3} (0.88)	13.8s	1.95s	1.91s	0.377s
akm_mobility_4	1.18×10^{-3} (0.63)	49.1s	5.93s	2.78s	0.348s
akm_mobility_5	1.02×10^{-3} (0.66)	52.4s (1/3)	5.95s	3.05s	0.346s
akm_mobility_6	4.63×10^{-4} (0.64)	failed (0/3)	7.33s	3.92s	0.329s

Note: AKM-style panel with 1M observations, one covariate, and worker, firm, and year fixed effects. Lower rows reduce worker mobility within a 10-period panel, so fewer workers connect different firms. A runtime followed by $\left(\frac{k}{3}\right)$ is based on k converged trials; `failed` means none of the three trials converged. Gap is defined as $1 - \rho_{WF}$ for the worker-firm pair; parentheses report the observation share of the component attaining the gap.

When mobility is high, `within` is slower than the MAP backends. As mobility declines, the worker-firm gap drops by more than two orders of magnitude to below 10^{-3}, and MAP runtimes increase. The `within` runtimes change little across the six designs, making it the fastest backend in the lowest-mobility rows.

Stronger sorting keeps more movers within firm groups, leaving fewer links between groups. MAP should slow as those links disappear.

Sorting benchmark ($n = 1$M).

Scenario	Gap (share)	rust-map	fixest	FEM.jl	within
akm_sorting_1	8.94×10^{-3} (0.96)	8.08s	1.36s	1.41s	0.401s
akm_sorting_2	3.58×10^{-3} (0.96)	14.7s	2.21s	1.74s	0.433s
akm_sorting_3	6.80×10^{-3} (0.96)	11.2s	2.08s	1.59s	0.444s
akm_sorting_4	4.38×10^{-3} (0.96)	15.6s	2.22s	1.69s	0.440s
akm_sorting_5	2.17×10^{-3} (0.96)	29.0s	3.07s	2.01s	0.434s

Note: AKM-style panel with 1M observations, one covariate, and worker, firm, and year fixed effects. Lower rows increase sorting among movers, so fewer movers connect firms in different groups. Gap is $1 - \rho_{WF}$ for the worker-firm pair; parentheses report the observation share of the component attaining the gap.

From the first sorting design to the last, the worker-firm gap falls by about a factor of four and MAP runtimes generally increase. The gap is not monotonic in the intermediate rows. The `within` runtimes change little across the five designs.

7.1.2. Standard Synthetic Benchmarks: fixest and Correia DGPs

The first family is the simple-versus-difficult benchmark data generating process from `fixest` (Bergé et al. 2026). Both designs use 10M observations, one covariate, and worker, firm, and year fixed effects. Both contain about the same number of worker-firm links. In the simple design, those links are spread broadly across workers and firms; in the difficult design, the matches are nearly nested. MAP should converge faster on the simple design. The table also reports legacy `torch-cuda` timings from the PyFixest benchmark suite.

Simple vs. difficult design (10M observations, 3 FE).

Design	Gap (share)	rust-map	fixest	FEM.jl	within	torch-cuda
simple (well-connected)	0.857 (1.00)	2.29s	2.65s	2.20s	12.0s	4.73s
difficult (near-nested)	1.67×10^{-7} (1.00)	347.9s	71.8s	28.8s	4.37s	8.73s

Note: CPU times are medians from three independently generated 10M-observation samples. The gap is computed on the first sample. `torch-cuda` values come from the PyFixest benchmark suite. Standalone `within` timings exclude regression overhead. Setup and solve take 7.18s and 2.27s on the simple design, versus 0.823s and 1.77s on the difficult design. Setup accounts for 77% and 32% of the respective totals.

On the simple design, both MAP backends converge quickly and `within` is slowest.

On the difficult design, where the worker-firm gap is 1.67×10^{-7}, `within` takes 4.37s, compared with 28.8s for `FEM.jl`, 71.8s for `fixest`, and 347.9s for unaccelerated `rust-map`.

We retain the historical `torch-cuda` values for reference, but do not compare them with the local CPU timings because the hardware and run details differ.

We also use the Correia synthetic datasets, which include complete, uniform, assortative, and path-like graphs.

Correia synthetic benchmarks.

Dataset	Gap (share)	rust-map	fixest	FEM.jl	within
synthetic-complete	1.00 (1.00)	0.079s	0.063s	0.036s	0.135s
synthetic-uniform-easy	0.651 (1.00)	0.118s	0.139s	0.060s	0.126s
synthetic-uniform-hard	0.184 (1.00)	0.269s	0.346s	0.197s	0.968s
synthetic-uniform-harder	0.0249 (1.00)	0.705s	1.19s	0.297s	0.540s
synthetic-assortative	1.33×10^{-3} (0.70)	13.9s	1.38s	1.24s	0.325s

Note: Medians over three runs. Gap denotes $1 - \rho$ for the `id1-id2` pair after the same singleton pruning; parentheses report the observation share of the component attaining the gap. In two-way models, smaller gaps are generally associated with slower MAP convergence. The table omits `synthetic-zigzag`. On this small path graph, `rust-map` and `fixest` reach their iteration caps, `FEM.jl` takes 0.727s, and `within` takes 0.019s.

Unlike the controlled AKM experiments, these datasets vary in both sample size and graph structure, so the runtimes reflect setup time as well as convergence. `within` is slower on the complete and easier uniform designs. On `synthetic-uniform-harder`, only `FEM.jl` is faster; on the assortative design, `within` has the shortest runtime.

7.1.3. Standard Real-Data Benchmarks: Correia Collection

Empirical graphs can contain units that appear in many observations, thin links between dense groups, many disconnected components, and interactions among more than two identifiers. The Correia real datasets contain these features.

Correia real-data benchmarks.

Dataset	Gap (share)	rust-map	fixest	FEM.jl	within
credit	0.402 (1.00)	0.150s	0.126s	0.097s	0.155s
soccer	0.889 (1.00)	0.018s	0.014s	0.013s	0.037s
enron	7.04×10^{-3} (0.99)	3.03s	0.519s	0.321s	0.357s
github	6.30×10^{-4} (0.32)	failed (0/3)	1.42s	1.48s	0.223s
patents	5.18×10^{-4} (0.95)	failed (0/3)	1.05s	0.623s	0.321s
workers	2.74×10^{-4} (0.63)	failed (0/3)	2.02s	1.52s	0.177s
schools	2.21×10^{-3} (1.00)	6.51s	0.818s	0.523s	0.162s
directors	5.12×10^{-4} (0.30)	failed (0/3)	0.293s	0.518s	0.233s

Note: Medians over three runs on singleton-dropped samples, as produced by the PyFixest benchmark suite. The gap is $1 - \rho$ for the id1-id2 pair after the same singleton pruning; parentheses report the observation share of the component attaining the gap. A small component share means that the reported gap applies to only part of the sample, as in directors.

Accelerated MAP is fastest on credit and soccer, where the gaps are large. In directors, the component with the smallest gap contains 30% of the observations, so that gap describes only part of the sample. On enron, within takes 0.357s and FEM.jl 0.321s. The factor-pair preconditioner is fastest on github, patents, workers, schools, and directors.

7.2. Poisson / PPML Benchmark

Generalized linear models with high-dimensional fixed effects are typically estimated by iteratively reweighted least squares (IRLS). Each IRLS step fits a weighted least squares problem in which the response and covariates are demeaned against the fixed effects, with weights that are updated between iterations (Correia et al. 2020; Stammann 2018). The demeaning operation is identical to the process described in the prior sections.

7.2.1. Preconditioner reuse across IRLS

The fixed-effect graph stays the same across IRLS steps, although the weights change. The implementation builds the preconditioner once per regression and reuses it at later steps. This may require more inner Krylov iterations, but it does not change the weighted least-squares solution if the inner solver converges. We use the simple and difficult fixest DGPs (Bergé et al. 2026) with 1M observations, one covariate, and worker, firm, and year fixed effects. The outcome is negative binomial with dispersion $\theta = 0.5$ and a log-linear conditional mean. PPML has the correct mean specification despite the non-Poisson variance. We compare R fixest's fepois, GLFixedEffectModels.jl, and two PyFixest fepois backends: the default unpreconditioned rust-map and the preconditioned within solver. All packages use a common cap of 100 outer IRLS iterations; their other stopping rules remain at package defaults.

Poisson benchmarks (1M observations, one covariate, worker-firm-year fixed effects).

Design	FE	rust-map	fixest	GLFEM.jl	within
simple (well-connected)	3	11.2s	5.72s	10.6s	8.71s
difficult (near-nested)	3	failed (0/3)	324.9s	174.5s	5.95s

Note: Medians over three full IRLS regression calls at $n = 1$M, one covariate, and three fixed effects. `fixest` is R `fixest::fepois`; `rust-map` and `within` are the PyFixest `fepois` routine with the unpreconditioned MAP backend and the factor-pair preconditioned solver, respectively; `GLFEM.jl` is `GLFixedEffectModels.jl`. `failed` indicates that all three `rust-map` trials reached the 10000-iteration MAP cap without converging. The well-connected and near-nested descriptions refer to the absorbed worker-firm graph.

On the simple design, all four backends finish in 5.72–11.2s. On the difficult design, `within` takes 5.95s, compared with 174.5s for `GLFEM.jl` and 324.9s for `fixest`; `rust-map` reaches its iteration cap.

7.3. Memory Use

MAP needs only the current residuals and per-level group sums. The factor-pair preconditioner also retains pair information between iterations. We measure peak resident set size (peak RSS) on the simple and difficult DGPs to quantify the additional memory. The comparison is limited to `within` and `rust-map` in the same Python package. Comparing peak RSS across Python, R, and Julia would also measure differences in language runtimes, data loading, garbage collection, and package internals. We run each backend in a separate process and read peak RSS from `ru_maxrss`.

Memory footprint (3 FE, one covariate).

Design	Gap (share)	`rust-map`	`within`
100K observations			
simple (well-connected)	0.857 (1.00)	316 MiB	368 MiB
difficult (near-nested)	1.30×10^{-3} (1.00)	312 MiB	351 MiB
1M observations			
simple (well-connected)	0.857 (1.00)	680 MiB	959 MiB
difficult (near-nested)	1.67×10^{-5} (1.00)	740 MiB	829 MiB

Note: Each cell is one run in an isolated Python process with one covariate and three fixed effects; the values are not medians. Gap denotes $1 - \rho_{WF}$ for the worker-firm pair.

The preconditioner adds 39–52 MiB at 100K observations and 89–279 MiB at 1M. The extra memory holds factor-pair co-occurrences, partition weights, and local approximate Cholesky factors.

7.4. Numerical Equivalence

We compare the new solver's coefficient estimate with the estimates returned by the other routines. Exact equality is not expected because the packages use different stopping criteria and tolerances.

The comparison uses the 100K-observation simple and difficult `fixest` DGPs. The worker-firm gap is 0.857 in the simple design and 1.30×10^{-3} in the difficult design. The table reports the coefficient on `x1` for all four backends.

Coefficient agreement (100K observations, 3 FE, one covariate).

Design	Backend	$\hat{\beta}_1$	Absolute difference
simple	`rust-map`	0.99871660	–
	`within`	0.99871660	8.9×10^{-16}
	`fixest`	0.99871660	1.1×10^{-14}
	`FEM.jl`	0.99871660	9.2×10^{-14}
difficult	`rust-map`	1.00321070	–
	`within`	1.00321085	1.5×10^{-7}
	`fixest`	1.00321085	1.5×10^{-7}
	`FEM.jl`	1.00321069	6.7×10^{-9}

Note: $\hat{\beta}_1$ is the slope coefficient on `x1`. The last column reports the absolute difference from `rust-map`. `within` is the PyFixest preconditioned Rust backend.

On the simple design, every backend agrees with `rust-map` to within 9.2×10^{-14}. On the difficult design, the largest difference is 1.5×10^{-7}. These checks cover one coefficient in two designs.

8. Software

The solver studied in this paper is available as open-source software through the `within` project (within Developers 2026). The computational core is implemented in Rust and exposed through Rust, Python, and R interfaces; each language binding invokes the same underlying solver. This shared core matters for reproducibility and adoption: the same algorithm can be called from Rust, Python, or R without reimplementation.

Interface	Package	Registry	Install
Rust	`within`	crates.io	`cargo add within`
Python	`within-py`	PyPI	`pip install within-py`
R	`within`	py-econometrics R-universe	`install.packages("within", repos = "https://py-econometrics.r-universe.dev")`

The Rust crate exposes the lower-level solver together with its configuration types. The Python and R packages provide solver-level `solve` and `solve_batch` APIs for residualizing one or several right-hand sides. The algorithm is also available as a demeaning backend for PyFixest (PyFixest Developers 2026). Here is a minimal Python example, in which we demean an outcome variable and two covariates against worker and firm fixed effects, before we run the FWL fit on the demeaned variables:

```python
import numpy as np
from within import solve_batch

# Worker and firm identifiers as a column-major uint32 array
n = 100_000
categories = np.asfortranarray(np.column_stack([
    np.random.randint(0, 5_000, n).astype(np.uint32),
    np.random.randint(0,   500, n).astype(np.uint32),
]))

# Outcome and covariates
beta = np.array([1.0, -2.0])
X = np.random.randn(n, 2)
y = X @ beta + np.random.randn(n)

# Residualize y and X jointly; the preconditioner is reused across columns
res = solve_batch(categories, np.column_stack([y, X]))
y_tilde, X_tilde = res.demeaned[:, 0], res.demeaned[:, 1:]

# FWL on the demeaned variables
beta_hat = np.linalg.lstsq(X_tilde, y_tilde, rcond=None)[0]
```

9. Conclusion

This paper introduces a factor-pair Schwarz preconditioner for residualizing high-dimensional fixed effects. The preconditioner uses pairwise co-occurrence tables, such as worker-firm match counts, while LSMR handles the remaining coupling among factors. On the well-connected designs in our benchmarks, setup takes longer than the iterations it saves, and MAP or diagonally preconditioned LSMR is faster. On poorly connected and near-nested designs, `within` is fastest in most of the lowest-gap cases and in the difficult PPML benchmark. Reusing the preconditioner across IRLS steps does not make it worthwhile on the well-connected PPML design, where `fixest` remains faster. Lower pairwise spectral gaps are associated with slower MAP convergence in these benchmarks, but they do not provide a numerical cutoff for choosing a solver.

Appendix A: Details of the Factor-Pair Schwarz Preconditioner

Two-Factor Block Inverse

For the two-factor block

$$G_2 = \begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix}, \tag{31}$$

let

$$S = G_{FF} - C_{(WF)'} G_{WW}^{-1} C_{WF}. \tag{32}$$

Standard block inversion gives

$$G_2^{-1} = \begin{pmatrix} G_{WW}^{-1} + G_{WW}^{-1} C_{WF} S^{-1} C_{(WF)'} G_{WW}^{-1} & -G_{WW}^{-1} C_{WF} S^{-1} \\ -S^{-1} C_{(WF)'} G_{WW}^{-1} & S^{-1} \end{pmatrix}. \tag{33}$$

Algorithm

Algorithm 1 gives the implementation corresponding to the construction summarized in Figure 4.

Algorithm 1. Factor-Pair Schwarz Preconditioner

Inputs

- Observation-level factor codes for Q absorbed dimensions.
- Diagonal weights W and a local solver configuration.
- Krylov residual r in coefficient space.

Preconditioner setup

- Enumerate all unordered factor pairs (q, r) with $q < r$.
- For each pair, build weighted count blocks G_{qq}, G_{rr} and the weighted cross-tabulation C_{qr}.
- Split the induced bipartite graph into connected components and create one Schwarz subdomain s per component.
- If fixed-effect level j appears in c_j subdomains, store the partition weight $\omega_j = \frac{1}{\sqrt{c_j}}$.
- For each subdomain, form $L_s = T_s G_s T_s$. If p_s is the number of local levels, set $\Pi_s = I_{p_s} - \mathbf{1}\frac{\mathbf{1}'}{p_s}$; multiplying by Π_s subtracts the component mean.
- Eliminate the larger factor block, leaving a reduced system on the smaller block. Solve a small reduced system by dense Cholesky. For a large system, approximate the fill-in cliques by sampling and apply randomized approximate Cholesky.

Krylov application

- Initialize $z = 0$.
- For each subdomain s, form $h_s = \tilde{D}_s R_s r$ and $b_s = \Pi_s T_s h_s$.
- Apply the stored local solver to b_s to obtain v_s, then set $u_s = T_s v_s$.
- Accumulate $z \leftarrow z + R_{s'} \tilde{D}_s u_s$.
- Return $z = M^{-1} r$.

References

Abowd, John M., Francis Kramarz, and David N. Margolis. 1999. "High Wage Workers and High Wage Firms." *Econometrica* 67 (2): 251–333. https://doi.org/10.1111/1468-0262.00020.

Arridge, Simon R., Marta M. Betcke, and Lauri Harhanen. 2014. "Iterated Preconditioned LSQR Method for Inverse Problems on Unstructured Grids." *Inverse Problems* 30 (7): 75009. https://doi.org/10.1088/0266-5611/30/7/075009.

Berge, Laurent. 2018. *Efficient Estimation of Maximum Likelihood Models with Multiple Fixed-Effects: The R Package Fenmlm.* Technical Report Nos. 2018–13.

Bergé, Laurent R., Kyle Butts, and Grant McDermott. 2026. "Fixest: A Fast and Feature-Rich Framework for Econometric Estimations in R.". https://doi.org/10.48550/arXiv.2601.21749.

Correia, Sergio. 2014. "Reghdfe: Stata Module to Perform Linear or Instrumental-Variable Regression Absorbing Any Number of High-Dimensional Fixed Effects.". https://github.com/sergiocorreia/reghdfe.

Correia, Sergio. 2017. *Linear Models with High-Dimensional Fixed Effects: An Efficient and Feasible Estimator.* Technical report. https://hdl.handle.net/10.2139/ssrn.3129010.

Correia, Sergio, Paulo Guimarães, and Tom Zylkin. 2020. "Fast Poisson Estimation with High-Dimensional Fixed Effects." *The Stata Journal* 20 (1): 95–115. https://doi.org/10.1177/1536867X20909691.

FixedEffectModels.jl Developers. 2026. "Fixedeffectmodels.jl: Fast Estimation of Linear Models with High Dimensional Categorical Variables.". https://github.com/FixedEffects/FixedEffectModels.jl.

Fong, David Chin-Lung, and Michael Saunders. 2011. "LSMR: An Iterative Algorithm for Sparse Least-Squares Problems." *SIAM Journal on Scientific Computing* 33 (5): 2950–71. https://doi.org/10.1137/10079687X.

Frisch, Ragnar, and Frederick V. Waugh. 1933. "Partial Time Regressions as Compared with Individual Trends." *Econometrica* 1 (4): 387–401. https://doi.org/10.2307/1907330.

Gao, Yuan, Rasmus Kyng, and Daniel A. Spielman. 2025. "AC(k): Robust Solution of Laplacian Equations by Randomized Approximate Cholesky Factorization." *SIAM Journal on Scientific Computing*,. https://rasmuskyng.com/papers/GKS25.pdf.

Gaure, Simen. 2013. "OLS with Multiple High Dimensional Category Variables." *Computational Statistics & Data Analysis* 66 : 8–18. https://doi.org/10.1016/j.csda.2013.03.024.

Goldsmith-Pinkham, Paul. 2026. *Tracking the Credibility Revolution across Fields.* Technical report.

Guimaraes, Paulo, and Pedro Portugal. 2010. "A Simple Feasible Procedure to Fit Models with High-Dimensional Fixed Effects." *The Stata Journal* 10 (4): 628–49. https://doi.org/10.1177/1536867X1101000406.

Irons, Bruce M., and Richard C. Tuck. 1969. "A Version of the Aitken Accelerator for Computer Iteration." *International Journal for Numerical Methods in Engineering* 1 (3): 275–77. https://doi.org/10.1002/nme.1620010306.

Lovell, Michael C. 1963. "Seasonal Adjustment of Economic Time Series and Multiple Regression Analysis." *Journal of the American Statistical Association* 58 (304): 993–1010. https://doi.org/10.1080/01621459.1963.10480682.

PyFixest Developers. 2026. “Pyfixest: Fast High-Dimensional Fixed Effects Regression in Python.”. https://github.com/py-econometrics/pyfixest.

Spielman, Daniel A., and Shang-Hua Teng. 2014. “Nearly Linear Time Algorithms for Preconditioning and Solving Symmetric, Diagonally Dominant Linear Systems.” *SIAM Journal on Matrix Analysis and Applications* 35 (3): 835–85. https://doi.org/10.1137/090771430.

Stammann, Amrei. 2018. “Fast and Feasible Estimation of Generalized Linear Models with High-Dimensional K-Way Fixed Effects.”. https://doi.org/10.48550/arXiv.1707.01815.

Toselli, Andrea, and Olof Widlund. 2005. *Domain Decomposition Methods: Algorithms and Theory*. Springer. https://doi.org/10.1007/b137868.

within Developers. 2026. “Within: High-Performance Fixed-Effects Solver for Econometric Panel Data.”. https://github.com/py-econometrics/within.

Xu, Jinchao. 1992. “Iterative Methods by Space Decomposition and Subspace Correction.” *SIAM Review* 34 (4): 581–613. https://doi.org/10.1137/1034116.

Yang, Mei, Gul Karaduman, and Ren-Cang Li. 2024. “Flexible Modified LSMR for Least Squares Problems.”.